entirely from function names. For example, consider a `send` function that accepts another function as the parameter:

```
send = (next) ->
  http.send next()

email = ->
```

The syntax is appealing:

```
send email (body 'Hi Agtron') to 'agtron@coffeescriptinaction.com'
```

Unfortunately, a DSL that uses function composition only is generally limited to a single line because of significant indentation. To get multiple lines, you might have to resort to using parentheses and line-continuing backslashes:

```
send email \
(body 'Hi Agtron!')\
(to 'agtron@coffeescriptinaction.com')
```

That makes DSLs based purely on the techniques of function composition a bit clunky in CoffeeScript. That said, there are other ways to use functions for DSLs more effectively, one of which you'll see in the examples that follow.

8.2.6. Constructing a DSL

The lack of syntactic noise in CoffeeScript has meant that people attracted to the notion of writing DSLs have flocked to the language. Here are some examples of domains for which DSLs have been created in CoffeeScript and an accompanying syntax example that's easily implemented in CoffeeScript. Implementing handlers for HTTP requests has been omitted here simply because you have, by now, seen it so many times that it would be rather uninteresting.

HTML